

1. Signal.c

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    printf("\nSIGINT received!\n");
    printf("Signal number = %d\n", sig);
}

int main()
{
    signal(SIGINT, handler);

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }

    return 0;
}
```

```
varshini@VarshiniBolla:~$ ./myshell
myshell> nano signal.c
myshell> gcc signal.c -o sl
myshell> ./sl
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

2. Sigterm

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");
}

int main()
{
    signal(SIGTERM, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for SIGTERM...\n");

    while (1)
    {
        sleep(2);
    }

    return 0;
}

```

```

myshell> nano sigterm.c
myshell> gcc sigterm.c -o sl
myshell> ./sl
Process ID: 805
Waiting for SIGTERM...

```

3. Sigchild

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void child_handler(int sig)
{
    printf("SIGCHLD received: Child has terminated\n");
    wait(NULL);
}

int main()
{
    signal(SIGCHLD, child_handler);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child is running...\n");
        sleep(2);
        printf("Child is exiting...\n");
    }
    else
    {
        printf("Parent is waiting...\n");
        sleep(5);
        printf("Parent finished.\n");
    }

    return 0;
}

```

```

myshell> nano sigchild.c
myshell> gcc sigchild.c -o sl
myshell> ./sl
Parent is waiting...
Child is running...
Child is exiting...
SIGCHLD received: Child has terminated
Parent finished.

```

4. Sigpipe

```

#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigpipe(int sig)
{
    printf("\nSIGPIPE received!\n");
    printf("The reading end of the pipe is closed.\n");
    exit(1);
}

int main()
{
    int fd[2];

    signal(SIGPIPE, handle_sigpipe);

    pipe(fd);

    printf("Pipe created.\n");

    // Close the reading end
    close(fd[0]);

    printf("Reading end closed.\n");

    printf("Trying to write to the pipe...\n");

    write(fd[1], "Hello", 5);

    printf("Write completed.\n");

    return 0;
}

```

```

myshell> gcc sigpipe.c -o sl
myshell> ./sl
Pipe created.
Reading end closed.
Trying to write to the pipe...

SIGPIPE received!
The reading end of the pipe is closed.

```

5. Sigsegv

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigsegv(int sig)
{
    printf("\nSIGSEGV received!\n");
    printf("Invalid memory access detected.\n");
    exit(1);
}

int main()
{
    signal(SIGSEGV, handle_sigsegv);

    printf("Program started.\n");

    int *p = NULL;

    printf("Trying to access invalid memory...\n");

    *p = 10;

    printf("Program completed.\n");

    return 0;
}
```

```
myshell> nano sigsegv.c
myshell> gcc sigsegv.c -o ls
myshell> ./ls
Program started.
Trying to access invalid memory...

SIGSEGV received!
Invalid memory access detected.
```